attributes like `department` belong.

10.2.2 Versioning and review

Treating policies like code also meant treating them like any other artifact in Git. Every change was made using a pull request and was then reviewed and approved by another engineer, resulting in a signed, versioned bundle. Developers could see exactly when and why a policy changed, and who approved it.

This history proved invaluable when auditors asked, “Who decided customers should be able to edit documents, and when?” The identity team didn’t have to reconstruct the decision from memory. They pulled up a six-month-old pull request that included reviewer comments explaining why edit rights were granted, along with a corresponding test that confirmed customers still couldn’t share documents externally.

Code reviews also enhanced policy quality. Reviewers didn’t just look for syntax errors — they posed business-level questions: Does this reflect the intent? Is this scope too broad? Over time, reviewing policy pull requests became a way for engineers, product managers, and even lawyers like Dan to share their perspectives on authorization.

10.2.3 Operational considerations

Centralized policy stores also provided ACME with operational visibility it had never experienced. Decision logs allowed them to finally answer the question, “Why was this request denied?” by referring directly to the relevant policy and condition. Metrics enabled them to measure latency and assess whether authorization was imposing unacceptable overhead.

Explainability proved to be one of the biggest wins. Before, developers would contact the identity team whenever they encountered an access denied error. Now, decision logs include reasons: Denied because `principal not in Project::alpha.members`. Here’s a snippet from the logs: